

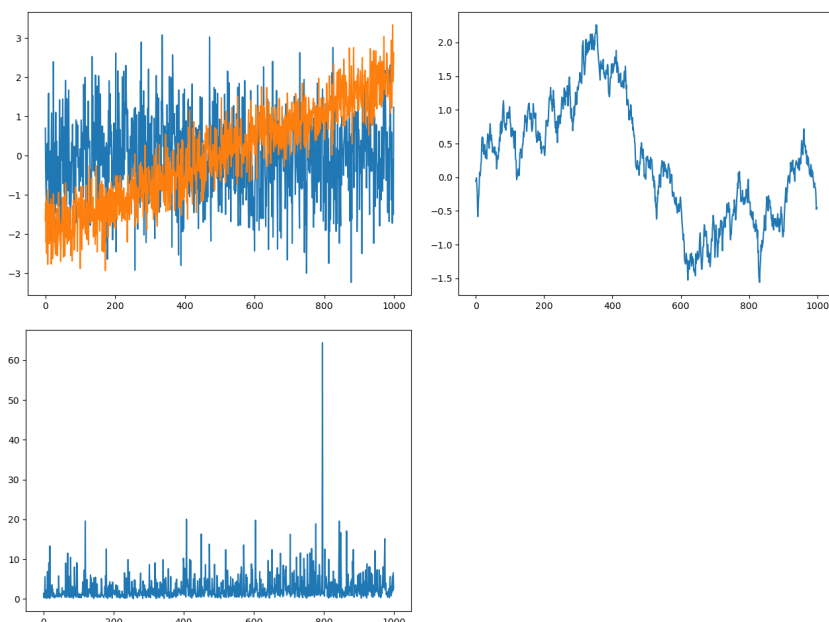
说明

首先，这篇文章从全文看，只在说明一个东西。就是说明rnn在时间序列上预测是要比一般的模型更加准确的。且，全篇没有实际的时间序列实验，所有的时间序列数据都是通过随机过程进行模拟得到的。由此，如果我们单独进行复现，基本就没有啥意义。因此，我们根据文章的思想，得到一个有趣的技术方案，即，从虚拟的数据中训练出一个在实际中可用的预测模型，且模型可用。该思想主要来自文章的2.4节，在这一节中，文章提出了一个从经验序列到实际序列的过程，我们的实操过程就是这一过程的实践。

三种过程

1. 一般的噪音过程
2. 均值回复过程
3. 马尔可夫转换过程

运行algs文件夹中的simulate.py文件，可以看到我们对文章三种过程的复现。



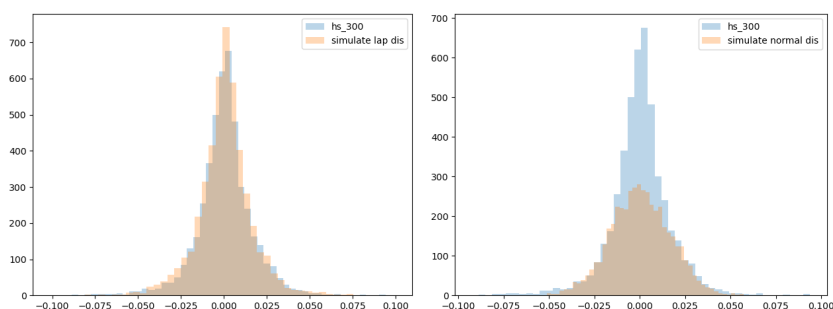
文中出现的各种结果都是可以通过调节参数来获得的。这里，我们对马尔可夫状态转换过程进行说明。

我们定义了一个新的过程，让最后所有的序列都不是通过固定的形式展现的。即使用马尔可夫转换与一般的噪音过程来定义：

首先，我们假设了三种状态， $\{-1, 0, 1\}$ ，我们定义三种状态的转换概率矩阵，从而可以随机生成一个状态序列，然后通过状态对收益率进行模拟。

注：文中是对价格的状态过程进行模拟，我们在实操的情况下，更多的是对收益率的接触，这样可以在不同指数下继续迁移应用，因此，我们描述的是更加普遍的情况。

文中三种状态都是假设残差是正态分布的条件下得到的显式结果，但是，我们发现收益率分布更加符合拉普拉斯分布。运行s1_exp_data.py可以得到如下分布图，明显的，沪深300的收益率分布更加贴合拉普拉斯分布。



则，我们可以通过状态序列来模拟每一次的收益率情况，且加入一个拉普拉斯的噪音。这样可以更加好的拟合真实的收益率。

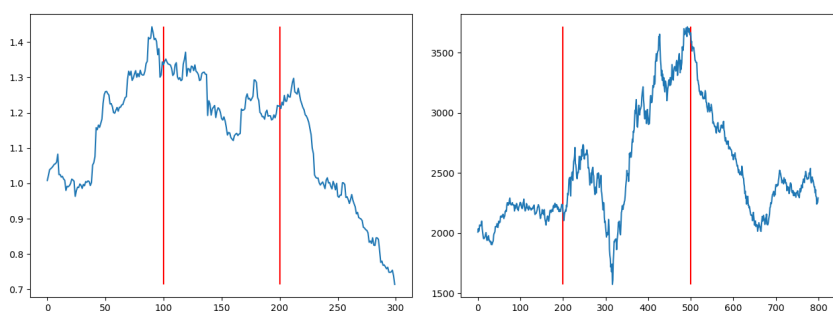
Ornstein-Uhlenbeck过程其实是一种均值回复的过程，性质可以参考[知乎链接](#)。就是说，我们还要使用一个均值回复的过程来更好的模型股票市场。

由此，我们结合文中提出了三种随机模拟状态，转换成：

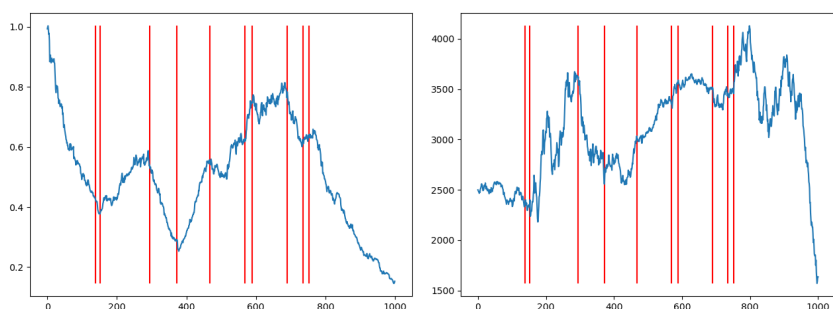
1. 马尔可夫转换链生成状态
2. 均值回复与噪音过程根据状态链生成收益率序列。

在生成的过程中，我们都是已知状态的。状态就可以作为训练时间序列中的标签。通过运行 `algs.fin_simulate.py` 可以模拟出不同的序列：

下图是自定义的一个状态为 $\{-1, 0, 1\}$ 的noisy过程与状态为 $\{0, 1, -1\}$ 的均值回复过程。



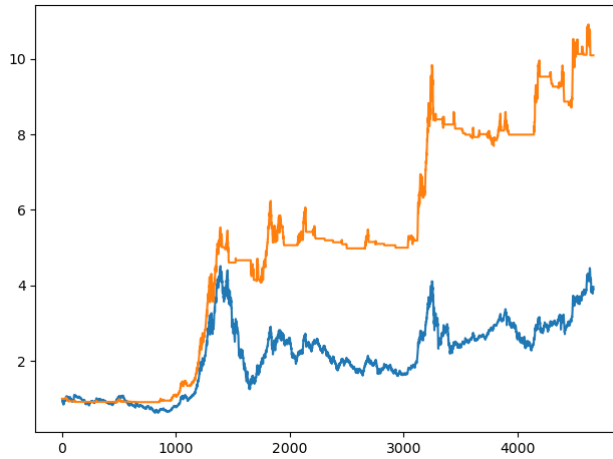
在加入马尔可夫转换状态概率示，随机生成如下的走势图：



实验过程

我们全程不使沪深300的数据，我们随机生成上面描述的序列，生成的序列自动的会生成label。此时的label是具有噪音的label。因此，我们其实学习到的是具有严重噪音的模型。刚好可以符合金融收益率的描述。

我们依照论文中，使用gru神经网络，每次生成16个随机的包括100个点的随机收益序列。在已知状态的情况下进行训练。我们训练1000次后，使用模型对沪深300进行预测，并对状态为1的点进行之后买入操作。



代码说明：

只需要运行s1_generate_data.py即可得到效果，其余的代码作为辅助，生成的图片在上面已经有说明。

补充说明

每个平台的优化策略有不同的问题，我已经通过设定种子来解决优化出现的问题。我先给出在linux与win10两个平台的优化过程：

```
1.083244800567627
1.0585395097732544
1.0530452728271484
1.1144524812698364
1.0561872720718384
1.093031406402588
1.1156054735183716
1.0863888263702393
1.1046644449234009
1.0897159576416016
1.098799228668213
1.0905094146728516
1.0939843654632568
1.0927350521087646
1.0916187763214111
1.0938165187835693
1.076155424118042
1.092934012413025
1.0686759948730469
1.0781508684158325
1.0879528522491455
1.1099677085876465
1.1084740161895752
1.0852787494659424
1.0684467554092407
1.107206106185913
1.107356071472168
1.0832446813583374
1.0585393905639648
1.0530452728271484
1.1144524812698364
1.056187391281128
1.093031406402588
1.1156052350997925
1.0863888263702393
1.1046642065048218
1.0897160768508911
1.0987993478775024
1.0905094146728516
1.0939843654632568
1.092734932899475
1.0916187763214111
1.0938165187835693
1.076155424118042
1.092934012413025
1.0686759948730469
1.078150987625122
1.0879528522491455
1.1099677085876465
1.1084740161895752
1.0852786302566528
1.0684467554092407
1.107206106185913
1.1073559522628784
1.0852264165878296
```

两个平台优化的前面结果基本一致。但是，在最后还是出现了优化上的不同结果：

0.8132108449935913	0.8241642713546753
0.7806408405303955	0.7900010347366333
0.765777051448822	0.7820938229560852
0.7663143873214722	0.7772648334503174
0.8518226146697998	0.8319239616394043
0.8260334730148315	0.8341194987297058
0.7940759658813477	0.7890281081199646
0.8275395035743713	0.8195658922195435
0.836384654045105	0.8521340489387512
0.8414095044136047	0.8546088933944702
0.7788137793540955	0.784512996673584
0.8583598136901855	0.835626482963562
0.7941505908966064	0.8063938021659851
0.770106852054596	0.784823477268219
0.8368675708770752	0.8423518538475037
0.8383750319480896	0.8411412835121155
0.7595486044883728	0.7779930233955383
0.8519221544265747	0.8528257012367249
0.833772599697113	0.8464386463165283
0.8304998874664307	0.838005781173706
0.7975131869316101	0.7893843054771423
0.802423894405365	0.8081803321838379
0.767809271812439	0.7841457724571228
0.8497398495674133	0.8435152173042297
0.8200441002845764	0.8194459676742554
0.8389216065406799	0.8410259485244751
0.8330082297325134	0.8437243103981018
0.8631471395492554	0.8462920188903809
0.7940320372581482	0.7970014810562134

in(2): | in(2):

因为，这篇论文只是说明rnn能够更好的做趋势预测。这个结论基本就是废话，我这边整个策略都是对文章的实用性引申，所以才会导致很多不成熟的想法。没想到会出现不同平台上差异巨大的问题。

因为整个模型的训练是一个噪音很大的训练场景，rnn模型作为神经网络，如果出现对某些噪音出现部分过拟合就会对整个结果产生一些影响。

第一个调整的点：代码的第8行，mk模型中，有一个参数，noisy_level用以控制噪音，该参数为空的情况下，默认为市场噪音为0.012，该参数是通过实际对hs_300模拟得到的。通过修改这个参数可以调节生成序列的噪音情况。从而训练模型。

第二个可以调整的点：代码127行，可以通过修改rnn模型的隐藏层size来决定模型的复杂度。一般来说，越复杂的模型对数据的拟合越强但是也越难训练，例如可以修改为GRU_model(4)变成一个简单一些模型。

第三个调整点：代码129行可以修改学习率和优化方法。可以尝试使用不同的优化器，后面已经对学习率做了一定的策略。

第四个调整点：代码的141与142行可以进行注释，表示是否要对参数进行正则化。

第五个调整点：代码的135行，可以使用更多次数的训练，比如将1000改成1500，136行，可以修改batch_size。

第六个调整点：代码的165行，可以使用best_model也可以使用model。两种使用都是有道理的。使用best模型是指使用训练过程中，训练损失最少的模型。用model表示使用训练最长时间的模型。因为数据都是随机的，历史最好并不能代表最好，但是绝对不差。而训练最久的理论上应该是最好的，但是训练过程往往会出现各种问题，并不一定有很好的表现。不过，我这边还是建议使用model而不是best_model。

造成差距太大的原因我猜测还是因为数据噪音太大，但是，噪音问题本身就是存在的金融问题。数据需要噪音来拟合金融市场，反过来噪音又造成模型的训练稳健性与效果。两者需要平衡点来解决。然而这个问题已经超越了文章本身，是一个更加普遍的问题。

关于做空的问题，我这边发现加入做空机制效果不佳，其原因是本身市场是一个涨的状态。所以，做空基本很难有收益。不过，考虑多空的代码我已经加入。

